

Assignment No: 1

- **Write a Program in python**

Q. To Find Average Of Marks Of 5 Subject.

```
# Program to calculate the average of marks of 5 subjects

# Taking input from the user for 5 subjects
marks = []
for i in range(1, 6):
    mark = float(input(f'Enter marks for subject {i}: '))
    marks.append(mark)

# Calculating the average
average = sum(marks) / len(marks)

# Displaying the result
print(f'\nThe average marks of 5 subjects is: {average:.2f}')
```

Output

```
Enter marks for subject 1: 90
Enter marks for subject 2: 98
Enter marks for subject 3: 89
Enter marks for subject 4: 78
Enter marks for subject 5: 88

The average marks of 5 subjects is: 88.60
```

Assignment No: 2

Q. To Find Sum Of 1st N Prime Numbers.

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

def sum_of_n_primes(N):
    count = 0
    num = 2
    prime_sum = 0

    while count < N:
        if is_prime(num):
            prime_sum += num
            count += 1
            num += 1

    return prime_sum

# Example usage
N = int(input("Enter the number of prime numbers to sum: "))
print("Sum of first", N, "prime numbers is:", sum_of_n_primes(N))
```

Output

```
Enter the number of prime numbers to sum: 5
Sum of first 5 prime numbers is: 28
```

Assignment No: 3

A) To Perform Linear Search.

```
def linear_search(arr, target):  
    for i in range(len(arr)):  
        if arr[i] == target:  
            return i  
    return -1
```

```
arr = [10, 20, 30, 40, 50]  
target = 30  
result = linear_search(arr, target)
```

```
print(f'Element found at index {result}' if result != -1 else  
      "Element not found")
```

Output

Element found at index 2

B) To Perform Binary Search.

```
def binary_search(arr, target):  
    left, right = 0, len(arr) - 1  
    while left <= right:  
        mid = (left + right) // 2  
        if arr[mid] == target:  
            return mid  
        if arr[mid] < target:  
            left = mid + 1  
        else:  
            right = mid - 1  
    return -1
```

```
arr = [10, 20, 30, 40, 50]  
target = 30  
result = binary_search(arr, target)
```

```
print("Element found at index", result if result != -1 else  
      "Element not found")
```

Output

```
Element found at index 2
```

Assignment No: 4

Q.To Find Factorial Of Given Number.

```
def factorial(n):  
    if n < 0:  
        return "Factorial is not defined for negative numbers"  
    elif n == 0 or n == 1:  
        return 1  
    else:  
        fact = 1  
        for i in range(2, n + 1):  
            fact *= i  
        return fact  
  
# Taking input from the user  
num = int(input("Enter a number: "))  
print(f"Factorial of {num} is {factorial(num)}")
```

Output

```
Enter a number: 4  
Factorial of 4 is 24
```

Assignment No: 5

Q.To Find Fibonacci Series Of Given Number.

```
def fibonacci(n):  
    a, b = 0, 1  
    for _ in range(n):  
        print(a, end=" ")  
        a, b = b, a + b  
  
num = int(input("Enter the number of terms: "))  
fibonacci(num)
```

Output

Enter the number of terms: 5

0 1 1 2 3

Assignment No: 6

Q. To Perform Sorting For Given List Using Quick Sort.

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[0] # Choosing the first element as pivot
    left = [x for x in arr[1:] if x < pivot]
    right = [x for x in arr[1:] if x >= pivot]
    return quick_sort(left) + [pivot] + quick_sort(right)

# Example usage
arr = [10, 7, 8, 9, 1, 5]
print("Sorted array:", quick_sort(arr))
```

Output

Sorted array: [1, 5, 7, 8, 9, 10]

Assignment No: 7

Q. To Perform Sorting For Given List Using Merge Sort.

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    while left and right:
        result.append(left.pop(0) if left[0] < right[0] else
right.pop(0))
    return result + left + right

# Example usage
arr = [10, 7, 8, 9, 1, 5]
print("Sorted array:", merge_sort(arr))
```

Output

Sorted array: [1, 5, 7, 8, 9, 10]

Assignment No: 8

Q. To Create a Stack And Perform Various Operation On Stack.

```
class Stack:
    def __init__(self):
        self.stack = []

    def is_empty(self):
        return len(self.stack) == 0

    def push(self, item):
        self.stack.append(item)
        print(f"Pushed {item} onto the stack.")

    def pop(self):
        if self.is_empty():
            print("Stack is empty. Cannot pop.")
            return None
        return self.stack.pop()

    def peek(self):
        if self.is_empty():
            print("Stack is empty. Nothing to peek.")
            return None
        return self.stack[-1]
```

```
def display(self):
    if self.is_empty():
        print("Stack is empty.")
    else:
        print("Stack contents:", self.stack)

# Example usage
if __name__ == "__main__":
    stack = Stack()
    stack.push(10)
    stack.push(20)
    stack.push(30)
    stack.display()
    print("Popped item:", stack.pop())
    print("Top item:", stack.peek())
    stack.display()
```

Output

```
Pushed 10 onto the stack.
Pushed 20 onto the stack.
Pushed 30 onto the stack.
Stack contents: [10, 20, 30]
Popped item: 30
Top item: 20
Stack contents: [10, 20]
```

Assignment No: 9

Q. To Create a Stack And Perform Various Operation On Queue.

```
from queue import Queue
```

```
# Stack Implementation
```

```
class Stack:
```

```
    def __init__(self):
```

```
        self.stack = []
```

```
    def push(self, item):
```

```
        self.stack.append(item)
```

```
    def pop(self):
```

```
        return self.stack.pop() if self.stack else "Stack is empty"
```

```
# Queue Implementation
```

```
class QueueOperations:
```

```
    def __init__(self):
```

```
        self.queue = Queue()
```

```
    def enqueue(self, item):
```

```
        self.queue.put(item)
```

```
def dequeue(self):  
    return self.queue.get() if not self.queue.empty() else  
    "Queue is empty"  
  
# Testing the Stack and Queue  
stack = Stack()  
queue = QueueOperations()  
  
# Stack Operations  
stack.push(10)  
stack.push(20)  
print("Stack Pop:", stack.pop())  
  
# Queue Operations  
queue.enqueue(30)  
queue.enqueue(40)  
print("Queue Dequeue:", queue.dequeue())
```

Output

Stack Pop: 20

Queue Dequeue: 30

Assignment No: 10

Q. To Create a Linked List And Perform Various Operation On Linked List.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")
```

Example usage:

```
ll = LinkedList()
```

```
ll.insert(10)
```

```
ll.insert(20)
```

```
ll.insert(30)
```

```
ll.display()
```

Output

```
30 -> 20 -> 10 -> None
```